```
    // Additional information regarding Math usage
    var infoPara = document.createElement('p');
    infoPara.textContent =
     'Student details include a randomly generated ID and a rounded percentage computed using Math methods.';
    document.body.appendChild(infoPara);
  };
 </script>
</head>

<body>
</body>
</html>
```

**7e. Write a program using string object properties and methods.**

The JavaScript String object is a global object that is used to store strings. A string is a sequence of letters, numbers, special characters and arithmetic values or combination of all. JavaScript provides a variety of built-in methods to work with strings — allowing you to extract, modify, search, and format text with ease.

String Properties

The following table lists the standard properties of the String object.

| Property | Description |
|---|---|
| length | Returns the length of a string. |
| prototype | Allows you to add new properties and methods to a String object. |

String Methods

The following table lists the standard methods of the String object.

| Method | Description |
|---|---|
| charAt() | Returns the character at the specified index. |
| charCodeAt() | Returns the Unicode of the character at the specified index. |
| concat() | Joins two or more strings, and returns a new string. |
| endsWith() | Checks whether a string ends with a specified substring. |
| fromCharCode() | Converts Unicode values to characters. |
| includes() | Checks whether a string contains the specified substring. |
| indexOf() | Returns the index of the first occurrence of the specified value in a string. |

| | |
|---|---|
| lastIndexOf() | Returns the index of the last occurrence of the specified value in a string. |
| localeCompare() | Compares two strings in the current locale. |
| match() | Matches a string against a regular expression, and returns an array of all matches. |
| repeat() | Returns a new string which contains the specified number of copies of the original string. |
| replace() | Replaces the occurrences of a string or pattern inside a string with another string, and return a new string without modifying the original string. |
| search() | Searches a string against a regular expression, and returns the index of the first match. |
| slice() | Extracts a portion of a string and returns it as a new string. |
| split() | Splits a string into an array of substrings. |
| startsWith() | Checks whether a string begins with a specified substring. |
| substr() | Extracts the part of a string between the start index and a number of characters after it. |
| substring() | Extracts the part of a string between the start and end indexes. |
| toLocaleLowerCase() | Converts a string to lowercase letters, according to host machine's current locale. |
| toLocaleUpperCase() | Converts a string to uppercase letters, according to host machine's current locale. |
| toLowerCase() | Converts a string to lowercase letters. |
| toString() | Returns a string representing the specified object. |
| toUpperCase() | Converts a string to uppercase letters. |
| trim() | Removes whitespace from both ends of a string. |
| valueOf() | Returns the primitive value of a String object. |

CODE:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <title>My College - Student Details</title>
  <script>
    // User-defined object constructor for Student
    function Student(name) {
      this.name = name;
      // Generate a random user id using Math.random() and Math.floor()
      this.id = "A" + Math.floor(Math.random() * 1000);
      this.displayDetails = function() {
              var studentDiv = document.createElement('div');
        studentDiv.innerHTML = '<h2>Student Details</h2>' +
                        '<p><strong>Name:</strong> ' + this.name + '</p>' +
                        '<p><strong>ID:</strong> ' + this.id + '</p>'
        // Append the div to the body using the pre-defined document object
        document.body.appendChild(studentDiv);
      }
    }

    // Run code after the document is fully loaded
    window.onload = function() {
      // Create a header element using a document method
      var header = document.createElement('h1');
      header.textContent = 'My College - Student Details';
      document.body.appendChild(header);

      var student1 = new Student("Arjun Kumar");
      // Display the student's details
      student1.displayDetails();

      // Additional information regarding String usage
      var infoPara = document.createElement('p');
      infoPara.textContent = 'This example demonstrates string object properties and methods applied to
student details.';
      document.body.appendChild(infoPara);
      // --------- String Object Properties and Methods Demonstration ---------
      // Section header for String Operations
      var stringHeader = document.createElement('h2');
      stringHeader.textContent = 'String Object Properties and Methods';
      document.body.appendChild(stringHeader);

      // Using the student name for various string operations
      var nameStr = student1.name; // "Arjun Kumar"
      // Display the length property of the string
      var lengthPara = document.createElement('p');
      lengthPara.textContent = 'Length of student name: ' + nameStr.length;
      document.body.appendChild(lengthPara);

      // Convert the string to uppercase
      var upperPara = document.createElement('p');
      upperPara.textContent = 'Uppercase: ' + nameStr.toUpperCase();
      document.body.appendChild(upperPara);

      // Convert the string to lowercase
      var lowerPara = document.createElement('p');
```

```
    lowerPara.textContent = 'Lowercase: ' + nameStr.toLowerCase();
    document.body.appendChild(lowerPara);

    // Extract a substring (first name) using substring() and indexOf()
    var firstName = nameStr.substring(0, nameStr.indexOf(" "));
    var substringPara = document.createElement('p');
    substringPara.textContent = 'First Name (using substring): ' + firstName;
    document.body.appendChild(substringPara);

    // Find the index of the space character in the name using indexOf()
    var indexPara = document.createElement('p');
    indexPara.textContent = 'Index of space in name: ' + nameStr.indexOf(" ");
    document.body.appendChild(indexPara);

    // Split the full name into an array of words using split()
    var splitName = nameStr.split(" ");
    var splitPara = document.createElement('p');
    splitPara.textContent = 'Split name (first and last): ' + splitName.join(" | ");
    document.body.appendChild(splitPara);

    // Display the first character of the name using charAt()
    var charPara = document.createElement('p');
    charPara.textContent = 'First character of the name: ' + nameStr.charAt(0);
    document.body.appendChild(charPara);
  }
 </script>
</head>
<body>
</body>
</html>
```

**7f. Write a program using regex object properties and methods.**

Regular Expressions

A Regular Expression is a sequence of characters that forms a search pattern.

Regex is a common shorthand for a regular expression.

JavaScript RegExp is an Object for handling Regular Expressions.

RegExp are be used for:

Text searching

Text replacing

Text validation

**Regex Section:** The regex demonstration section works on the student's name:

test(): Checks if the name contains only letters and spaces.

match(): Extracts the first word from the name.

search(): Finds the index of the first space.

replace(): Removes all spaces from the name.

exec(): Executes the regex and returns the first match.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>My College - Student Details</title>
 <script>
  // User-defined object constructor for Student
  function Student(name, branch) {
```

```
    this.name = name;
    // Generate a random user id using Math.random() and Math.floor()
    this.id = "A" + Math.floor(Math.random() * 1000);
    this.branch = branch;

    // Method to display student details using document methods
    this.displayDetails = function() {
      // Create a div element to hold the student details
      var studentDiv = document.createElement('div');
      studentDiv.innerHTML = '<h2>Student Details</h2>' +
                        '<p><strong>Name:</strong> ' + this.name + '</p>' +
                        '<p><strong>ID:</strong> ' + this.id + '</p>' +
                        '<p><strong>Branch:</strong> ' + this.branch + '</p>';
      // Append the div to the body using the document object
      document.body.appendChild(studentDiv);
    }
  }

  // Run code after the document is fully loaded
  window.onload = function() {
    // Create a header element using a document method
    var header = document.createElement('h1');
    header.textContent = 'My College - Student Details';
    document.body.appendChild(header);

    // Create an instance of the Student object (marks and percentage dropped)
    var student1 = new Student("Arjun Kumar", "Computer Science");
    student1.displayDetails();

    // --------- Regex Object Properties and Methods Demonstration ---------
    var regexHeader = document.createElement('h2');
    regexHeader.textContent = 'Regex Object Properties and Methods';
    document.body.appendChild(regexHeader);

    // Define a regular expression to validate that the student's name contains only letters and spaces
    var nameRegex = /^[A-Za-z\s]+$/;

    // Use test() to check if the name matches the regex
    var isNameValid = nameRegex.test(student1.name);
    var testPara = document.createElement('p');
    testPara.textContent = 'Using test(): The student name "' + student1.name + '" is ' + (isNameValid ?
'valid' : 'invalid') + '.';
    document.body.appendChild(testPara);

    // Use match() to extract the first word (first name) from the student name
    var matchResult = student1.name.match(/[A-Z][a-z]+/);
    var matchPara = document.createElement('p');
    matchPara.textContent = 'Using match(): The first word in the name is "' + (matchResult ?
matchResult[0] : 'Not Found') + '".';
    document.body.appendChild(matchPara);

    // Use search() to find the index of the first space in the student name
    var searchIndex = student1.name.search(/\s/);
    var searchPara = document.createElement('p');
```

```
    searchPara.textContent = 'Using search(): The index of the first space in the name is ' + searchIndex
+ '.';
    document.body.appendChild(searchPara);
    // Use replace() to remove all spaces from the student name
    var nameWithoutSpaces = student1.name.replace(/\s/g, "");
    var replacePara = document.createElement('p');
    replacePara.textContent = 'Using replace(): The name without spaces is "' + nameWithoutSpaces +
'".';
    document.body.appendChild(replacePara);
    // Use exec() to execute a search for a match (similar to test but returns an array)
    var execResult = nameRegex.exec(student1.name);
    var execPara = document.createElement('p');
    execPara.textContent = 'Using exec(): The result is ' + (execResult ? execResult[0] : 'null') + '.';
    document.body.appendChild(execPara);
  }
 </script>
</head>
<body>
</body>
</html>
```

**7g. Write a program using date object properties and methods.**

In JavaScript, the Date object is a built-in object that allows you to work with dates and times. It provides a variety of methods to handle dates, compare them, manipulate them, and format them.

The Date object in JavaScript represents a single point in time. It provides various functionalities to work with dates and times like creating a Date object in several ways, and it allows you to perform operations like extracting parts of the date (year, month, day), comparing dates, and manipulating dates.

Date Methods

The following table lists the standard methods of the Date object.

| Method | Description |
|---|---|
| getDate() | Returns the day of the month (from 1-31). |
| getDay() | Returns the day of the week (from 0-6). |
| getFullYear() | Returns the year (four digits). |
| getHours() | Returns the hour (from 0-23). |
| getMilliseconds() | Returns the milliseconds (from 0-999). |
| getMinutes() | Returns the minutes (from 0-59). |
| getMonth() | Returns the month (from 0-11). |
| getSeconds() | Returns the seconds (from 0-59). |
| getTime() | Returns the number of milliseconds since midnight Jan 1, 1970. |

| | |
|---|---|
| getTimezoneOffset() | Returns the time difference between UTC time and local time, in minutes. |
| getUTCDate() | Returns the day of the month, according to universal time (from 1-31). |
| getUTCDay() | Returns the day of the week, according to universal time (from 0-6). |
| getUTCFullYear() | Returns the year, according to universal time. |
| getUTCHours() | Returns the hour, according to universal time (from 0-23). |
| getUTCMilliseconds() | Returns the milliseconds, according to universal time (from 0-999) |
| getUTCMinutes() | Returns the minutes, according to universal time (from 0-59). |
| getUTCMonth() | Returns the month, according to universal time (from 0-11). |
| getUTCSeconds() | Returns the seconds, according to universal time (from 0-59). |
| getYear() | Deprecated. Use the getFullYear() method instead. |
| now() | Returns the number of milliseconds since midnight Jan 1, 1970. |
| parse() | Parses a date string and returns the number of milliseconds since Jan 1, 1970. |
| setDate() | Sets the day of the month of a date object. |
| setFullYear() | Sets the full year of a date object. |
| setHours() | Sets the hours of a date object. |
| setMilliseconds() | Sets the milliseconds of a date object. |
| setMinutes() | Set the minutes of a date object. |
| setMonth() | Sets the month of a date object. |
| setSeconds() | Sets the seconds of a date object. |

**CODE:**

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<title>My College - Student Details</title>
<script>
  // User-defined object constructor for Student
  function Student(name, branch, dob, joinYear) {
    this.name = name;
    // Generate a random user id using Math.random() and Math.floor()
    this.id = "A" + Math.floor(Math.random() * 1000);
    this.branch = branch;
    // Store date of birth (as a Date object) and the year of joining
    this.dob = new Date(dob); // dob should be in a format like "1999-03-15"
    this.joinYear = joinYear;
    // Calculate the age using the Date object
    var today = new Date();
    var age = today.getFullYear() - this.dob.getFullYear();
    var m = today.getMonth() - this.dob.getMonth();
    if (m < 0 || (m === 0 && today.getDate() < this.dob.getDate())) {
        age--;
    }
    this.age = age;
    // Method to display student details using document methods
    this.displayDetails = function() {
      var studentDiv = document.createElement('div');
      studentDiv.innerHTML = '<h2>Student Details</h2>' +
                             '<p><strong>Name:</strong> ' + this.name + '</p>' +
                             '<p><strong>ID:</strong> ' + this.id + '</p>' +
                             '<p><strong>Branch:</strong> ' + this.branch + '</p>' +
                             '<p><strong>Date of Birth:</strong> ' + this.dob.toDateString() + '</p>' +
                             '<p><strong>Age:</strong> ' + this.age + '</p>' +
                             '<p><strong>Year of Joining:</strong> ' + this.joinYear + '</p>';
      document.body.appendChild(studentDiv);
    }
  }
  // Run code after the document is fully loaded
  window.onload = function() {
    // Create a header element using document methods
    var header = document.createElement('h1');
    header.textContent = 'My College - Student Details';
    document.body.appendChild(header);
    // Create an instance of the Student object with a date of birth and year of joining
    var student1 = new Student("Arjun Kumar", "Computer Science", "1999-03-15", 2018);
    student1.displayDetails();
    // --------- Date Object Properties and Methods Demonstration ---------
    var dateHeader = document.createElement('h2');
    dateHeader.textContent = 'Date Object Properties and Methods';
    document.body.appendChild(dateHeader);
    // Create a Date object for the current date and time
    var currentDate = new Date();
    // Display the current date and time using toLocaleString()
    var currentDatePara = document.createElement('p');
    currentDatePara.textContent = 'Current Date and Time: ' + currentDate.toLocaleString();
    document.body.appendChild(currentDatePara);
    // Display individual properties of the current date
    var currentYearPara = document.createElement('p');
    currentYearPara.textContent = 'Current Year: ' + currentDate.getFullYear();
```

```
    document.body.appendChild(currentYearPara);
    var currentMonthPara = document.createElement('p');
    // getMonth() returns month index (0-11), add 1 for human-readable format
    currentMonthPara.textContent = 'Current Month: ' + (currentDate.getMonth() + 1);
    document.body.appendChild(currentMonthPara);
    var currentDayPara = document.createElement('p');
    currentDayPara.textContent = 'Current Day: ' + currentDate.getDate();
    document.body.appendChild(currentDayPara);
    var currentHoursPara = document.createElement('p');
    currentHoursPara.textContent = 'Current Hour: ' + currentDate.getHours();
    document.body.appendChild(currentHoursPara);
    var currentMinutesPara = document.createElement('p');
    currentMinutesPara.textContent = 'Current Minutes: ' + currentDate.getMinutes();
    document.body.appendChild(currentMinutesPara);
    var currentSecondsPara = document.createElement('p');
    currentSecondsPara.textContent = 'Current Seconds: ' + currentDate.getSeconds();
    document.body.appendChild(currentSecondsPara);
  }
 </script>
</head>
<body>
</body>
</html>
```

**7h. Write a program to explain user-defined object by using properties, methods, accessors, constructors and display.**

JavaScript is an object-oriented programming language that allows the creation of user-defined objects. An object is a collection of related data and functions grouped together as a single unit. User-defined objects help in organizing code, improving readability, and supporting modular programming.

A JavaScript object can consist of the following components:

**1. Properties**

Properties are variables that belong to an object. They store data related to that object. Properties represent the characteristics or attributes of the object. Each property consists of a name and a value.

Properties help in maintaining the state of an object and allow access to stored data through object references.

**2. Methods**

Methods are functions defined inside an object. They describe the behavior or actions that the object can perform. Methods can access and manipulate the object's properties using the this keyword.

Methods provide functionality and allow objects to perform specific operations related to their data.

**3. Accessors (Getters and Setters)**

Accessors are special methods used to control access to object properties.

- A **Getter** method is used to retrieve the value of a property.
- A **Setter** method is used to update or modify the value of a property.

Accessors improve data security and encapsulation by controlling how properties are accessed or modified. They allow validation and controlled data manipulation.

**4. Constructors**

A constructor is a special function used to initialize object properties when a new object is created. It acts as a blueprint for creating multiple objects with similar structure and behavior.

Constructors ensure that each object created has its own set of properties and methods. Objects are created using the constructor function along with the new keyword.

**CODE:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My College - Student Details</title>
```

```
  <script>
    // User-defined object constructor for Student
    function Student(firstName, lastName, branch) {
      // Properties
      this.firstName = firstName;
      this.lastName = lastName;
      this.branch = branch;
      // Generate a random user ID using Math.random() and Math.floor()
      this.id = "A" + Math.floor(Math.random() * 1000);

      // Accessor: Define a getter and setter for fullName
      Object.defineProperty(this, 'fullName', {
        get: function() {
          return this.firstName + " " + this.lastName;
        },
        set: function(name) {
          var parts = name.split(" ");
          this.firstName = parts[0];
          this.lastName = parts[1] || "";
        },
        enumerable: true,
        configurable: true
      });

      // Method: Display student details on the page using document methods
      this.displayDetails = function() {
        var studentDiv = document.createElement('div');
        studentDiv.innerHTML = '<h2>Student Details</h2>' +
                               '<p><strong>ID:</strong> ' + this.id + '</p>' +
                               '<p><strong>Full Name:</strong> ' + this.fullName + '</p>' +
                               '<p><strong>Branch:</strong> ' + this.branch + '</p>';
        document.body.appendChild(studentDiv);
      }
    }

    // Run the code after the document is fully loaded
    window.onload = function() {
      // Create an instance of the Student object using the constructor
      var student1 = new Student("Arjun", "Kumar", "Computer Science");

      // Display the student details initially
      student1.displayDetails();

      // Demonstrate the accessor: update the full name using the setter
      var updatePara = document.createElement('p');
      updatePara.textContent = "Updating full name using the accessor...";
      document.body.appendChild(updatePara);
      // Update the fullName property (this will update firstName and lastName)
      student1.fullName = "Rahul Sharma";
      // Display the updated student details
      student1.displayDetails();
    }
  </script>
</head>
<body>
```

</body>
</html>